Projeto de E-Commerce — Documento de Escopo Completo (Versão Final Revisada)

> **Propósito:** Documento de especificação completo, profissional e detalhado para orientar o desenvolvimento de um projeto de e-commerce full stack utilizando **React (Vite)** no frontend, **Express + TypeScript** no backend e **PostgreSQL** como banco principal - **armazenamento local de imagens** no servidor. Inclui requisitos funcionais, não-funcionais, arquitetura, segurança, endpoints, banco de dados, fluxo de reservas de estoque, logging/monitoramento e backlog de implementação.

---

1. Resumo Executivo

O objetivo é construir uma aplicação de e-commerce de porte médio, adequada para portfólio, com complexidade suficiente para demonstrar domínio em arquitetura, backend, frontend e boas práticas de software. O projeto inclui:

- Catálogo, carrinho, checkout e pedidos;
- Painel administrativo com métricas e análises;
- Sistema de avaliação (reviews + ratings);
- Busca avançada (full-text PostgreSQL);
- Controle de estoque com reservas e expiração via cron jobs;
- RBAC (Customer, Admin, SuperAdmin);
- Otimização de imagens usando **sharp**, salvando localmente (pasta do projeto);
- Sistema opcional de recomendações;
- Sistema opcional de notificações (e-mail e/ou WebSocket);
- Logging e monitoramento (arquivo + logs estruturados + health checks).

---

2. Objetivos e Critérios de Sucesso

Objetivos

- Demonstrar aplicação realista com fluxo completo de e-commerce.
- Garantir arquitetura limpa e modular, seguindo padrões de produção.
- Implementar segurança, autenticação, autorização e boas práticas.
- Entrega de dashboards administrativos e métricas.

Critérios de sucesso

- Usuário consegue navegar, comprar e visualizar seus pedidos.
- Admin controla produtos, usuários e pedidos.
- RBAC funcionando corretamente (403 para ações não permitidas).
- Estoque consistente com sistema de reserva + expiração automática.
- Projeto rodando em ambiente local via scripts, sem dependências externas.

---

3. Lista de Funcionalidades

Funcionalidades Obrigatórias

- Autenticação, cadastro e recuperação de senha.
- Catálogo de produtos com categorias e variantes.
- CRUD de produtos no admin.
- Carrinho persistente.
- Checkout (Stripe API em modo teste).
- Reservas de estoque + expiração via cron.
- Sistema de reviews e notas.
- Busca avançada (PostgreSQL full-text search).
- Painel administrativo com analytics e métricas.
- RBAC completo.
- Otimização e armazenamento local de imagens.

Funcionalidades Opcionais

- Sistema simples de recomendações.
- Notificações (e-mail / WebSocket).

---

4. Stack Tecnológico

Frontend

- **Zod** — validação compartilhada entre frontend e backend, garantindo consistência de tipos e schemas.

- **React (Vite)** + TypeScript — rápido, modular, leve e ótimo para portfólio.

- **React Router** — navegação client-side, controle de rotas públicas/privadas e layout.
- **React Query** — estado de servidor com cache, SSR-friendly e otimizado.
- **Zustand** — gerenciamento de estado global simples, rápido e baseado em store minimalista
- **MUI ou Shadcn/Tailwind** — UI moderna e responsiva.
- **react-hook-form** — formulários robustos e performáticos.
- **Recharts** — gráficos para analytics no admin.
- Autenticação via **JWT + Cookies HttpOnly** — padrão seguro e profissional.

Backend

- **Zod** — validação de entrada, schemas centralizados e segurança adicional para todas as rotas.

- **Node.js + Express + TypeScript** — minimalista e ideal para demonstrar arquitetura limpa.

- **Prisma ORM** — migrações, tipagem forte e DX excelente.
- **PostgreSQL** — full-text search nativo, robusto e confiável.
- **node-cron** — tarefas recorrentes (expirar reservas, limpeza etc.).
- **sharp** — otimização de imagens locais.
- **Nodemailer (SMTP)** — envio de e-mails sem serviços externos.

• **Stripe** — pagamento realista sem complexidade excessiva.
• **Express-Rate-Limit** — proteção contra abuso e brute-force (adicionado conforme solicitado).

Observações Técnicas

• Todo upload permanece **local** (pasta /uploads), nada de cloud.
• Estrutura do backend baseada em **camadas (controllers → services → repositories)**.
• Suporte a WebSockets opcional (para notificações).

5. Arquitetura de Alto Nível. Arquitetura de Alto Nível

Browser ⇄ API Express ⇄ PostgreSQL ↳ Sistema de Arquivos (uploads / imagens) ↳ Stripe Webhooks Cron Jobs (node-cron) ⇄ Banco + Sistema de Arquivos

Características:

• API stateless
• Imagens manipuladas e salvas localmente
• Cron jobs: expiração de reservas, limpeza de arquivos, envio assíncrono de e-mails

---

6. Modelo de Dados (conceitual — estilo Prisma)

Abaixo segue uma versão mais descritiva e contextualizada do modelo de dados. O objetivo é explicar **o motivo de cada tabela**, **o papel nos fluxos do e-commerce** e **como elas se relacionam**, mantendo o formato limpo e direto.

6.1. User

Representa qualquer pessoa com conta no sistema. O campo **role** define permissões (customer, admin, superadmin).

• id (PK)
• name, email (único)
• password_hash
• role
• created_at, updated_at

> **Contexto:** Essencial para autenticação, autorização e associação de carrinho, pedidos, reviews e notificações.

6.2. Address

Endereços de entrega e cobrança.

• id (PK)
• user_id (FK → User)
• campos de endereço (street, city, etc.)

> **Contexto:** Um usuário pode ter vários endereços; usado no checkout.

6.3. Product

Produto base visível no catálogo.

- id (PK)
- title, description
- base_price
- main_image
- published
- created_at, updated_at

Contexto: Não contém estoque nem atributos como cor/tamanho — isso pertence às variantes.

6.4. ProductVariant

Cada combinação de atributos vendável (ex.: cor preta + tamanho M).

- id (PK)
- product_id (FK → Product)
- sku (único)
- attributes (JSON)
- stock (estoque real)
- price_override (opcional)

Contexto: Estoque é controlado por variante, permitindo granularidade e lógica realista.

6.5. Category & ProductCategory

Categorias e a tabela de junção.

Category

- id (PK), name, slug

ProductCategory

- id (PK)
- product_id (FK)
- category_id (FK)

Contexto: Um produto pode estar em várias categorias (N\:N). Essencial para navegação e SEO.

6.6. Image

Todas as imagens extras de um produto (galeria).

- id (PK)
- product_id (FK)
- path (local)
- width, height
- optimized (boolean)

Contexto: Suporta múltiplas imagens por produto, mantendo metadados já otimizados.

6.7. Cart & CartItem

Implementação do carrinho persistente.

Cart

- id (PK)
- user_id (FK)
- created_at, updated_at

CartItem

- id (PK)
- cart_id (FK)
- variant_id (FK)
- quantity
- unit_price_snapshot

Contexto: Preço do item fica salvo na época da adição, evitando inconsistências caso o produto mude de preço.

6.8. Order & OrderItem

Pedido finalizado.

Order

- id (PK)
- user_id (FK)
- status (pending/paid/cancelled)
- total_amount
- payment_intent_id
- created_at

OrderItem

- id (PK)

- order_id (FK)
- variant_id
- quantity, unit_price

Contexto: Snapshot completo da compra; não deve depender dos dados atuais do produto.

6.9. Review

Avaliações reais de usuários.

- id
- product_id (FK)
- user_id (FK)
- rating (1–5)
- title, comment
- moderated
- created_at

Contexto: Controla spam via moderação e exige pedido prévio.

6.10. InventoryReservation

Reservas temporárias para checkout.

- id
- variant_id (FK)
- quantity
- expires_at
- created_at

Contexto: Evita overselling mantendo reservas paralelas ao estoque real. Removidas por cron job.

6.11. Notification (opcional)

Notificações internas e de sistema.

- id
- user_id
- type
- message
- read
- created_at

Contexto: Usado com WebSocket ou e-mail.

6.12. AuditLog

Rastreia ações administrativas.

- id
- actor_id (FK)
- action
- entity_type
- entity_id
- timestamp

> **Contexto:** Essencial para segurança e diagnósticos.

---

7. Endpoints Principais (Resumo)

Autenticação

- POST /auth/register
- POST /auth/login
- POST /auth/logout
- POST /auth/refresh

Produtos

- GET /products
- GET /products/\:id
- POST /products (admin)
- PUT /products/\:id (admin)
- DELETE /products/\:id (admin)
- GET /search?q=...

Carrinho

- GET /cart
- POST /cart/items
- PUT /cart/items/\:id
- DELETE /cart/items/\:id

Checkout

- POST /checkout/create-intent
- POST /checkout/confirm (via webhook Stripe)

Pedidos

- GET /orders
- GET /orders/\:id
- POST /orders/\:id/cancel
- GET /admin/orders

Reviews

- POST /products/\:id/reviews
- GET /products/\:id/reviews

Admin

- GET /admin/analytics
- GET /admin/users

---

8. Fluxo de Reserva de Estoque (sem — usando PostgreSQL + cron)

1) Início do Checkout

- Verifica estoque disponível (estoque_total - reservas_ativas).
- Cria **InventoryReservation** com expireAt = agora + 15 minutos.
- Garante atomicidade com transação ou `SELECT ... FOR UPDATE`.

2) Pagamento Confirmado (Stripe Webhook)

- Confirma reserva.
- Atualiza estoque real.
- Cria Order + OrderItems.
- Remove reservas relacionadas.

3) Expiração Automática via Cron

- Job roda a cada 5 min:
- Identifica reservas expiradas.
- Remove registros.
- Libera estoque implícito.

---

9. Segurança (Padrão de Mercado)

Autenticação

- Senhas hash com bcrypt ou argon2.
- Cookies HttpOnly + Secure para refresh token.
- Access token de curta duração.

Autorização (RBAC)

- Controle de roles: customer, admin, superadmin.
- Middlewares específicos por rota sensível.

Proteção de Dados

- Entrada validada com Zod/Joi.

- Sanitização de payloads.
- SQL Injection mitigado via Prisma.
- CSRF mitigado usando HttpOnly + SameSite.

Uploads

- Validação estrita de tipo e tamanho.
- Otimização com sharp.
- Restrição de caminhos para evitar path traversal.

Rate Limiting (novo)

Implementação simples usando **express-rate-limit**, com estratégias:

- **/auth/login** → limite mais rígido (ex.: 5 req/10 min) para evitar brute force.
- **/auth/register** → prevenir spam.
- **/admin/** → limites adicionais devido à sensibilidade.

  Opcionalmente registrar tentativas excedidas em AuditLog.

Boas Práticas Gerais

- Helmet configurado.
- Remoção de headers sensíveis.
- Logs estruturados para incidentes.